

Advanced Object-Oriented Programming

SWEN1 | BIF3

Table of Contents

- Advanced Concepts in Object-Oriented Programming



Advanced Concepts in Object-Oriented Programming

OOP kinds – Class-based vs Prototype-based

Class-based programming,
e.g.: Java and C#

```
public class Foo {
  public string Name { get; set; }
  public int One { get; set; }
  public int Two { get; set; }
}

public class Bar {
  public int Three { get; set; }
}

var foo = new Foo() { Name="foo", One=1, Two=2 };
var bar = new Bar() { Two=2, Three=3 };

Console.WriteLine( bar.One );   // resolves to 0
Console.WriteLine( bar.Three ); // resolves to 3

Console.WriteLine( bar.Two ); // resolves to 2
Console.WriteLine( bar.Name ); // resolves to null
Console.WriteLine( foo.Name ); // resolves to foo
```

Prototype-based programming,
e.g.: JavaScript

```
var foo = {name: "foo", one: 1, two: 2};
var bar = {two: "two", three: 3};

Object.setPrototypeOf(bar, foo);
// foo is now the prototype of bar.

// If we try to access foo's properties from bar
// from now on, we'll succeed.
bar.one // Resolves to 1.

// The child object's properties are also accessible.
bar.three // Resolves to 3.

//Own properties shadow prototype properties
bar.two; // Resolves to "two"
bar.name; // unaffected, resolves to "foo"
foo.name; // Resolves to "foo"
```

see https://en.wikipedia.org/wiki/Prototype-based_programming

Constructor

- a construct like a method used to initialize an object in all complexity levels.
- returns a reference to an object (do not specify a return type).
- method named is the class name
- it is possible to chain calls to multiple constructors (must be the first statement)
- constructors of a base class is always called before the actual constructor
- in case no constructor is defined a default constructor (no arguments) is added by the compiler

Destructor

- helps to “destroy” an object.
- especially necessary if resources need to be closed manually
  - OS handlers
  - free unmanaged resources
  - free unmanaged memory

Object Instanciation (C#, Java)

By instantiating an object the corresponding RAM is requested (**heap**) and the object "lives" physically in memory. The constructor is called (initiates the object) and in the code-block creating the object the caller can store a reference to the object. The reference points to a position in memory (heap). Many references to a single object can be created. In case no reference is provided to an object the **garbage collector** cleans the corresponding memory and free-s the object. (see: DataTypes, value vs. reference types)

Assigning a reference to a value indicating that you "currently don't know the value" is possible using **null**, but also consider Tony Hoare's Billion Dollar Mistake. (In most common languages it is an illegal operation to access members of null.)

Goals of OOP

- keep the object in a valid state
- may refer to objects of the real world
- keep corresponding methods near to the data (so method can use its object-data)
  - design your classes highly cohesive
- loose coupled
- size: single responsibility principle as part of SOLID
  (Robert C Martin: *“There should never be more than one reason for a class to change.”*
  and later: *“A module should be responsible to one, and only one, actor.”*)
- cohesion degree how elements (data and methods) relate to each other (usage) inside a class.

If done right...

Users MUST

- consider each piece as a part of the whole system (dependencies)
- should NOT be concerned with its implementation (only outer interface)
- NOT be allowed to fool around with (manipulate directly) the variables inside (check SOLID's OCP)
- force the user to use the prepared and safe operations the class provides to manipulate the objects

Namespaces (C#) and Packages (Java)

- group items logically (mostly classes)
- handles name collisions properly
- some languages
  - enforce mapping of namespaces with folder structure (Java) and
  - some don’t (C#, C++, Python)

Generics

Generics let you adapt a...

- method
- class
- structure
- interface
  to use the datatype you force him to use without the need to cast.

These generics can have further constraints.

Generics are used in Collections all over the place.

Delegates in C#

- encapsulating the method's call from caller
- can be used to call one or multiple methods asynchronously (especially UI)
- most common delegate-types are already prepared in the framework:
  - Action (void, takes up to 16 parameters)
  - Function (returns a value, takes up to 16 parameters)
  - Predicate (returns bool, takes exactly 1 parameter)
  - EventHandler
- multi-cast delegates possible (assign n methods)
- delegate operator (rarely used)
- lambda expressions

```
Ation<int> action = (num) => Console.WriteLine($"action executed {num}");
action+= (num) => Console.WriteLine($"I like {num}");

action?.Invoke(123);
```

Events in C#

- in C# the event handlers are called synchronously
- events are a concrete implementation of multi-cast delegates
- EventArgs are handed in using Generics

```
public event EventHandler<EventArgs> DataAvailable;
```

Delegates in Java

- concept does not really exist
- the same functionality achieved using class implementations

```
public interface SomeBehaviour { void SomeFunction(); }

public class MyBehaviour implements SomeBehaviour {
  public void SomeFunction() {
    // ...
  }
}

SomeBehaviour someBehaviour = MyBehaviour();
someBehaviour.someFunction();

someBehaviour = new SomeBehaviour {
  @Override
  public void SomeFunction() {
    // ...
  }
}
```

Events in Java

- create an interface (e.g.: HelloListener)
- create an addListener method to add listeners (like eventHandlers)
- in case an event fires call all the listeners

```
interface HelloListener { void someoneSaidHello(); }

// Someone who says "hello"
public class Initiater {
  private List<HelloListener> listeners = new ArrayList<>();
  public void AddListener(HelloListener toAdd) { listener.add(toAdd); }

  public void sayHello() {
    for (HelloListener hl : listeners ) { hl.someoneSaidHello();     }
  }

  public static void main(String[] args) {
    Initiater initiater = new Initiater();
    initiater.addListener( new HelloListener() {
      @Override public void someoneSaidHello() { System.out.println("Hi ho..."); }
    } );
    initiater.sayHello();
  }
}
```

Garbage Collection

... in **C#:**
in case an object is not referenced anymore the garbage collector removes it. Hereby a lot of optimization takes place when to remove the objects.

- `IDisposable` (`using` -keyword)
- `System.GC` (don't try this at home. . .)

... in **Java**:

- `Disposable`, `Closable`, `AutoClosable`

Structs in C#

In comparison to classes they:

- focus on data
- value type (stack)
- same member except default constructor
- no inheritance except inheritance from System.Object
- can implement interfaces

Instantiation:

- with custom constructor
- with default constructor (even if another is implemented)

Exceptions

- Occurrence of an exception (anomalous / exceptional condition) during execution
- Breaks normal flow
- jumps to exception handler
- `try / catch / finally` -blocks (order of the catch blocks count)

Exception Handling

- base type: `System.Exception` (C#) / `java.lang.Throwable` (Java)
- in Java distinguish between runtime- and checked- exceptions
- after a `try`-block multiple `catch`-blocks handle different exception types (check order, general catches below specific)
- an exception that can not be handled inside the method can be “rethrown”
- a `finally`-block can be defined to execute code in any case
- in C# conditions can be added (keyword `when`)
- When using reference-types take care about references to `null`
  --> lead to `NullReference-Exception` (C#) / `NullPointerException` (Java)

Implementing Exception Handling

Java:

```java
try {
  String[] array = new String[2];
  array[4] = "an ArrayIndexOutOfBoundsException";
  int number = Integer.parseInt("Not a number");
  // a NumberFormatException
}
catch ( ArrayIndexOutOfBoundsException
        | NumberFormatException e) {
  System.out.println("Caught exception: " +
    e.getMessage());
}
finally {
  System.out.println("This will always be executed.");
}
```

C#:

```csharp
try {
  string[] array = new string[2];
  array[4] = "an IndexOutOfRangeException";
  int number = int.parse("Not a number");
  // a FormatException
}
catch (IndexOutOfRangeException e)
{
  Console.WriteLine("Caught: " +    e.Message);
}
catch (FormatException e)
{
  Console.WriteLine("Caught : " + e.Message);
}
finally
{
  Console.WriteLine("This will always be executed.");
}
```

Exception Throwing & Custom Exceptions

Java:

```java
public class CustomException extends Exception {
  public CustomException(String message) {
    super(message);
  }
}

...

public void methodThatThrows(int value)
  throws CustomException
{
  if (value < 0) {
    throw new CustomException(
      "Value cannot be negative");
  }
  System.out.println("Value is: " + value);
}
```

C#:

```csharp
public class CustomException : Exception {
  public CustomException(string msg) : base(msg)
  {
  }
}

...

public void MethodThatThrows(int value)
{
  if (value < 0)
  {
    throw new CustomException(
      "Value cannot be negative");
  }
  Console.WriteLine("Value is: " + value);
}
```